

Designated-initializers for Base Classes

Document #: P2287R1
Date: 2021-02-15
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[3 Proposal](#)

[3.1 Naming the base classes](#)

[3.2 Directly name base-class members](#)

[3.3 Lookup of base classes](#)

[4 Wording](#)

[5 References](#)

§ 1 Revision History

[[P2287R0](#)] proposed a single syntax for a *designated-initializer* that identifies a base class. Based on a reflector suggestion from Matthias Stearn, this revision extends the syntax to allow the brace-elision version of *designated-initializer*: allow naming indirect non-static data members as well. Also actually correctly targeting EWG this time.

§ 2 Introduction

[[P0017R1](#)] extended aggregates to allow an aggregate to have a base class. [[P0329R4](#)] gave us designated initializers, which allow for much more expressive and functional initialization of aggregates. However, the two do not mix: a designated initializer can currently only refer to a direct non-static data members. This means that if I have a type like:

```
struct A {  
    int a;  
};  
  
struct B : A {  
    int b;  
};
```

While I can initialize an A like `A{.a=1}`, I cannot designated-initialize B. An attempt like `B{{1}, .b=2}` runs afoul of the rule that the initializers must either be all designated or none designated. But there is

currently no way to designate the base class here.

Which means that my only options for initializing a B are to fall-back to regular aggregate initialization and write either `B{{1}, 2}` or `B{1, 2}`. Neither are especially satisfactory.

§ 3 Proposal

This paper proposes extending designated initialization syntax to include both the ability to name base classes and also the ability to name base class members. In short, based on the above declarations of A and B, this proposal allows all of the following declarations:

```
B{{1}, 2}           // already valid in C++17
B{1, 2}            // already valid in C++17
B{:A={.a=1}, b=2}  // proposed
B{:A{.a=1}, b=2}   // proposed
B{:A{1}, .b{2}}    // proposed
B{.a=1, .b=2}      // proposed
B{.a{1}, .b{2}}    // proposed
```

§ 3.1 Naming the base classes

The tricky part here is: how do we name the A base class of B in the *designated-initializer-list*? While non-static data members have *identifiers*, base classes can be much more complicated. They can be qualified names, they can have template arguments, etc. We also do not actually have a way to name the A base class subobject of a B today — the only way to get there is via a cast. This means there's no corresponding consistent syntax to choose along with the `.` that we already have.

Daveed Vandevorde makes the suggestion that we can use `:` to introduce a *class-or-decltype* that names a base class (that is the grammar term we use when introducing a base class). This would allow the following initialization syntax:

```
B{:A={.a=1}, .b=2}
```

Using a `:` mimics the way we introduce base classes in class directions and is otherwise unambiguous with the rest of the *designated-initializer* syntax. It can also prepare the parser for the fact that a more complicated name might be coming.

This paper does not change any of the other existing designated-initialization rules: the initializers must still be all designated or none designated, and the designators must be in order. I'm simply extending the order being matched against with all the base classes. That is, while `B{:A={.a=1}, .b=2}` would be a valid way to initialize a B, `B{.b=2, :A={.a=1}}` is ill-formed (out of order), as is `B{{.a=1}, .b=2}` (some designated but not all).

This generalizes to more complex aggregates like:

```
template <typename T> struct C { T val; };
struct D : C<int>, C<char> { };

D{:C<int>={.val=1}, :C<char>={.val='x'}};
```

Which provides protection against `D{'x', 1}` which compiles fine today but probably isn't what was desired.

§ 3.2 Directly name base-class members

In most use-cases, while an aggregate may have base classes (which may themselves have further base classes), all the names of all the members all the way down will be distinct. The initial example here is just such a case: `B` has two members, `a` and `b`. We already have brace elision, which allows for `B{1, 2}`. But the designated initializer model is a lot safer: you have to name the members, so you can't really do the wrong thing. So it seems like a logic extension to support `B{.a=1, .b=2}`.

We still preserve all the rules around designated initializers: members need to be named *in order*. It's just that we remove the restriction that all the members have to be direct members of the class we're initializing.

That is:

```
B{:A={.a=1}, .b=2};           // fully explicit
B{:A={1}, .b=2};             // ok
B{.a=1, .b=2};               // ok

B{.b=2, .a=1};               // error: out of order
B{.b=2, :A{1}};              // error: out of order
B{:A{.a=1}, .a=2, .b=3};     // error: both A and a are named
```

The last row there has to be ill-formed. If an indirect member is named, you can't also name any class that contains it. But there's also another issue to consider here:

```
class string {
    char* begin;
    char* end;
    char* capacity;
public:
    string();
    string(char const*);
};

struct D : string {
    int index;
};

D{"hello", 42};               // already ok
D{:string("hello"), .index=42}; // ok

char storage[10];
D{.begin=storage, .end=storage, .capacity=storage+10, .index=17}; // definitely error
```

This obviously has to be an error. `string` isn't an aggregate, so this feature can't give us magic aggregate-initialization powers. So the rule for naming indirect members has to be both that:

1. the indirect member is not a (direct or indirect) member of any base class also named in the designated-initializer-list, and

2. the indirect member is not a (direct or indirect) member of any base class that is not an aggregate.

I'd also have to handle the case where the same named member appears in multiple base classes:

```
struct X { int x; };
struct Y : X { int x; };
Y{x=1};
```

This is already valid today and clearly needs to remain valid: the direct `Y::x` member is initialized to `1` and the base `X::x` member is initialized to `0`. The rule for *which* member is named would have to be adjusted to make it clear that it's based on class member access. `Y::x` names the `Y` member, not the `x` member, so that's the one that gets initialized.

Likewise, this nonsense:

```
struct X { int x; };
struct Y { int x; };
struct Z : X, Y { };
Z{x=1};
```

would be ill-formed on the basis that `Z::x` is ambiguous.

§ 3.3 Lookup of base classes

One other thing we need to consider is how we look up base classes exactly. With regular designated initializers, they're just the names of direct members and there's only one way to name them. Not much to talk about. But with base classes, we have an *injected-class-name* too, so we have to ask the question:

```
template <typename T> struct C { T val; };
struct D : C<int> { };

D{C<int>{.val=0}}; // proposed okay, C<int> is a base class
D{C{.val=1}};     // how about this?
```

From within the scope of `D`, we can use `C` to identify the base class `C<int>`. Likewise `D::C` names that type. Can we do this externally? Designated initializers already sort of look like they're from within the class. This seems like it should probably apply to base classes as well.

But in order for `:C` to find `D::C<int>::C` there, we'd have to say that lookup is in the context of the body of `D`. But then what if we have:

```
namespace N { template <typename T> struct C { }; }
struct D : N::C<int> { };
using C = N::C<double>;

D{:C{}};
```

Does `:C` refer to the *injected-class-name* of the `C<int>` base class of `D` (and thus be well-formed), or does `:C` refer to the alias `C` (and thus be ill-formed, since `N::C<double>` is not a base class of `D`)? Arguably, we are initializing a `D` so looking up from the context of `D` is a sensible rule (and is consistent with the rules we have for `x.operator T()` looking up `T` in the context of `x` 6.5.3 [\[basic.lookup.unqual\]/5](#)).

We just need to make sure that we don't do that kind of lookup for a *decltype-specifier* as a base class, since that probably makes very little sense to consider from the class' context.

§ 4 Wording

Add a rule for looking up unqualified names used in designators in 6.5.3 [\[basic.lookup.unqual\]](#):

- 5 An unqualified name that is a component name ([*expr.prim.id.unqual*]) of a *type-specifier* or *ptr-operator* of a *conversion-type-id* is looked up in the same fashion as the *conversion-function-id* in which it appears. If that lookup finds nothing, it undergoes unqualified name lookup; in each case, only names that denote types or templates whose specializations are types are considered.

- * An unqualified name that appears as the *class-or-decltype* in a *designator* in a *designated-initializer-list* ([*dcl.init.general*]) is looked up in the same fashion as if it were a *conversion-function-id* in the same context. If that lookup finds nothing, it undergoes unqualified name lookup; in each case, only names that denote types or templates whose specializations are types are considered.

[Example:

```
namespace N {
    struct A { int a; };
    template <typename T> struct B : T { int b; };
}

using C = N::A;

N::B<A>{A{.a=1}, .b=2}; // ok, lookup for A in N::B<A> finds the injected-
                        class-name A
N::B<A>{C{.a=1}, .b=2}; // ok, lookup for A in N::B<A> finds nothing,
                        regular unqualified lookup finds C

-end example]
```

Change the grammar of a *designator* in 9.4.1 [\[dcl.init.general\]](#)/1. Technically this allows a *designated-initializer-list* like `{.a=1, :A={}}` which we could forbid grammatically, but that seems more complicated than simply extending the ordering rule to forbid it (which has to be done anyway).

```
designator:
    . identifier
+      : class-or-decltype
```

Extend 9.4.1 [\[dcl.init.general\]](#)/18:

- 18 The same *identifier* shall not appear in multiple designators of a *designated-initializer-list*.
The same *class-or-decltype* shall not appear in multiple designators of a *designated-initializer-list*.

Extend 9.4.2 [\[dcl.init.aggr\]](#)/3.1:

- (3.1) If the initializer list is a *designated-initializer-list*, the aggregate shall be of class type, the *identifier* in each designator shall name a ~~direct~~ non-static data member of the class, *the class-or-decltype* in each designator shall name a base class of the class, and the explicitly initialized elements of the aggregate are the elements that are, or contain, those members. If any *identifier* in a designator names a (direct or indirect) non-static data member of a base class that is either named by a *class-or-decltype* in a different designator or that is not an aggregate, the initialization is ill-formed.

Extend 9.4.5 [dcl.init.list]/3.1:

- (3.1) If the *braced-init-list* contains a *designated-initializer-list*, τ shall be an aggregate class. The ordered *class-or-decltypes* and *identifiers* in the designators of the *designated-initializer-list* shall form a subsequence of the ordered *base classes of τ* and *identifiers* in the ~~direct~~ non-static data members of τ . Aggregate initialization is performed ([dcl.init.aggr]). [Example 2:

```
struct A { int x; int y; int z; };
A a{.y = 2, .x = 1};           // error: designator order does not
    match declaration order
A b{.x = 1, .z = 2};           // OK, b.y initialized to 0

+ struct B : A { int q; };
+ B c{.q = 3, :A{}};           // error: designator order does not
    match declaration order
+ B d{:A{}, .q = 3};           // OK, d.x, d.y, and d.z all
    initialized to 0
+ B e{.x = 1, .q = 3};         // OK, e.y and e.z initialized to 0
+ B f{:A{}, .x = 1, .q = 3};   // error: x is a member of A, which
    also appears in the designated-initializer-list

+ struct NonAggr { int na; NonAggr(int); };
+ struct D : NonAggr { int d; };
+ D g{:NonAggr{1}, .d=2};      // OK
+ D h{.na=1, .d=2};           // error: na is a member of a class
    that is not an aggregate
```

— end example]

§ 5 References

[P0017R1] Oleg Smolsky. 2015-10-24. Extension to aggregate initialization.

<https://wg21.link/p0017r1>

[P0329R4] Tim Shen, Richard Smith. 2017-07-12. Designated Initialization Wording.

<https://wg21.link/p0329r4>

[P2287R0] Barry Revzin. 2021-01-21. Designated-initializers for base classes.

<https://wg21.link/p2287r0>